МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D-графики. Правильный октаэдр

Выполнил: Андрианов А.Е.
Группа: М8О-312БВ-24
Преподаватель: Шангин Даниил Денисович

Москва, 2026

Условие

Цель работы — изучить построение простых трёхмерных объектов, проецирование на плоскость экрана и аффинные преобразования. Разработать интерактивное приложение на C++ с использованием Vulkan, GLFW и ImGui.

Вариант 2: правильный октаэдр. Требуется обеспечить корректную отрисовку фигуры в реальном времени и изменение её преобразований через пользовательский интерфейс.

Выполнены дополнительные задания: переключение перспективной и ортографической проекций; перенос, поворот и масштабирование по трём осям; движение и вращение по сложной траектории с паузой и настройкой скорости; изменение цвета; процедурная окраска вершин; несколько объектов с независимыми наборами дескрипторов.

Метод решения

Геометрия объекта. Октаэдр задан шестью точками: (±1, 0, 0), (0, ±1, 0), (0, 0, ±1). Восемь граней образованы треугольниками, соединяющими по одной вершине каждой оси. Длина каждого из 12 рёбер равна $\sqrt{2}$, поэтому грани являются равносторонними треугольниками. Вершины хранятся в vertex buffer, а 24 индекса задают восемь треугольников.

Преобразования. Для каждой модели вычисляется $M = T \cdot Rz \cdot Ry \cdot Rx \cdot S$. При использовании векторов-столбцов сначала применяется масштаб S, затем повороты и перенос T. Положение вершины после проекции определяется как $pclip = P \cdot V \cdot M \cdot plocal$. Матрица вида задаёт положение наблюдателя вдоль оси Z.

Проекции. Перспективная матрица зависит от угла обзора и соотношения сторон области сцены. Ортографическая матрица сохраняет видимый размер объектов при изменении расстояния до них. Используется диапазон глубины Vulkan [0, 1]; знак координаты Y корректируется в матрице проекции.

Метод решения (продолжение)

Ресурсы Vulkan и шейдеры. В initialize() создаются буферы геометрии, uniform buffers, VkDescriptorSetLayout, VkDescriptorPool, VkDescriptorSet, VkPipelineLayout и графический VkPipeline. Память выделяется через Vulkan Memory Allocator. Каждый объект использует свой uniform buffer с матрицами и цветом; геометрия общая для всех копий.

Обработка кадра. Функция update() формирует интерфейс ImGui и обновляет время анимации. После ожидания fence предыдущего кадра render() записывает данные в mapped buffers и вызывает vmaFlushAllocation. Затем записываются команды начала render pass, установки viewport и scissor, привязки pipeline, vertex/index buffers и descriptor set. Каждый видимый объект рисуется vkCmdDrawIndexed.

Окраска и освещение. Процедурный цвет вершины рассчитывается по локальной позиции: $c = (p + 1) / 2$. В vertex shader он умножается на цвет, выбранный в интерфейсе. Fragment shader использует интерполированный цвет и простое освещение плоских граней. Нормаль грани определяется по производным мировой позиции фрагмента.

Анимация. Первый объект перемещается по периодической траектории: $x = x_0 + r \sin(t)$, $y = y_0 + h \sin(2t)$, $z = z_0 + 0{,}5r \sin(3t)$. Одновременно углы вращения меняются пропорционально времени. Скорость задаёт темп накопления фазы t; пауза сохраняет её значение. Параметры остальных объектов редактируются независимо.

Завершение работы. В shutdown() выполняется ожидание завершения GPU, затем освобождаются pipeline, layout, descriptor pool и буферы. Наборы дескрипторов освобождаются вместе с pool. Это исключает использование ресурсов после их уничтожения.

Результаты

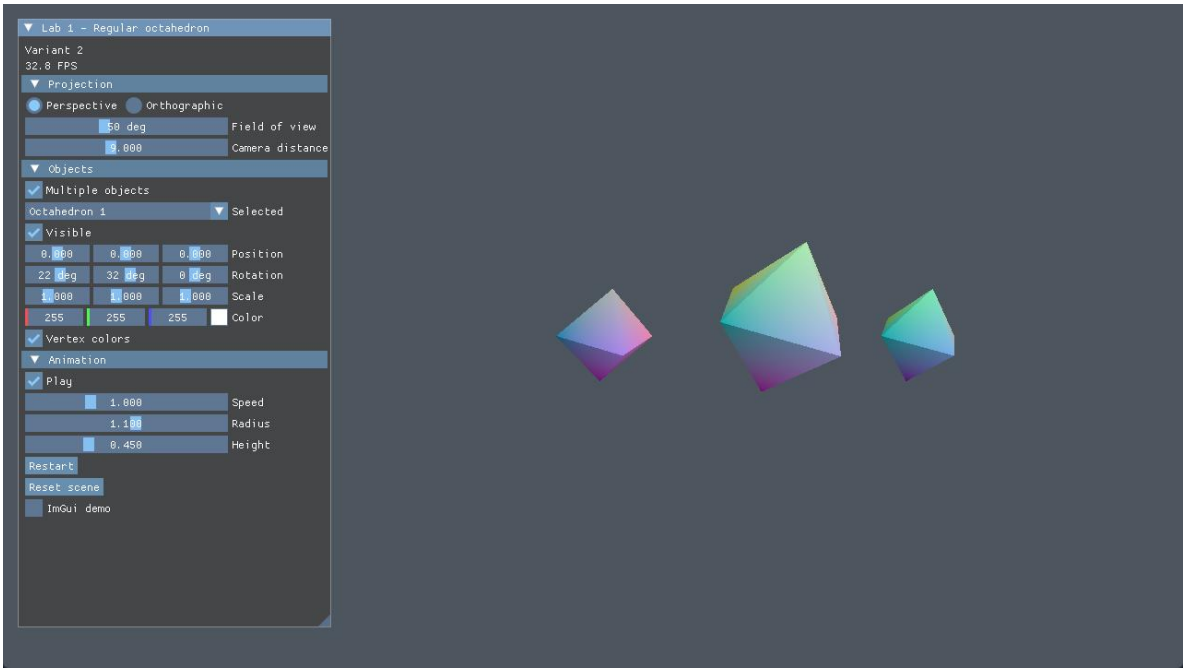


Рисунок 1 — Перспективная проекция, процедурная окраска и анимация первого объекта. Три октаэдра используют общую геометрию и независимые параметры.

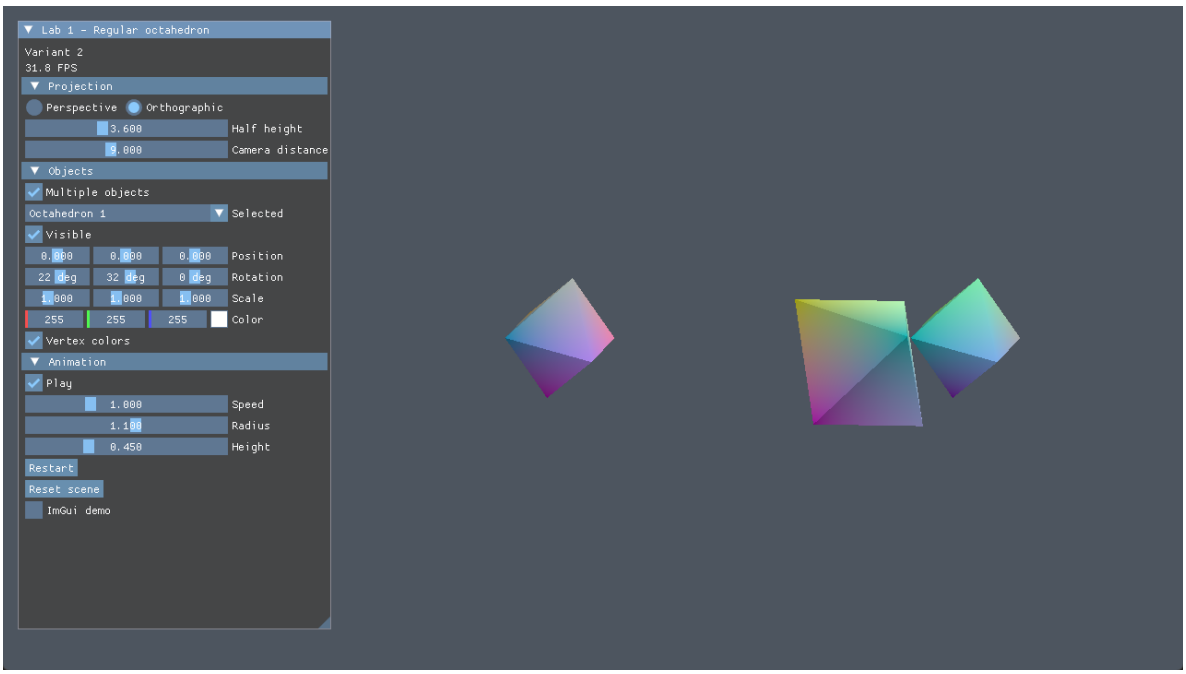


Рисунок 2 — Ортографическая проекция. Переключение типа проекции выполняется во время работы приложения без пересоздания геометрии.

Результаты (продолжение)

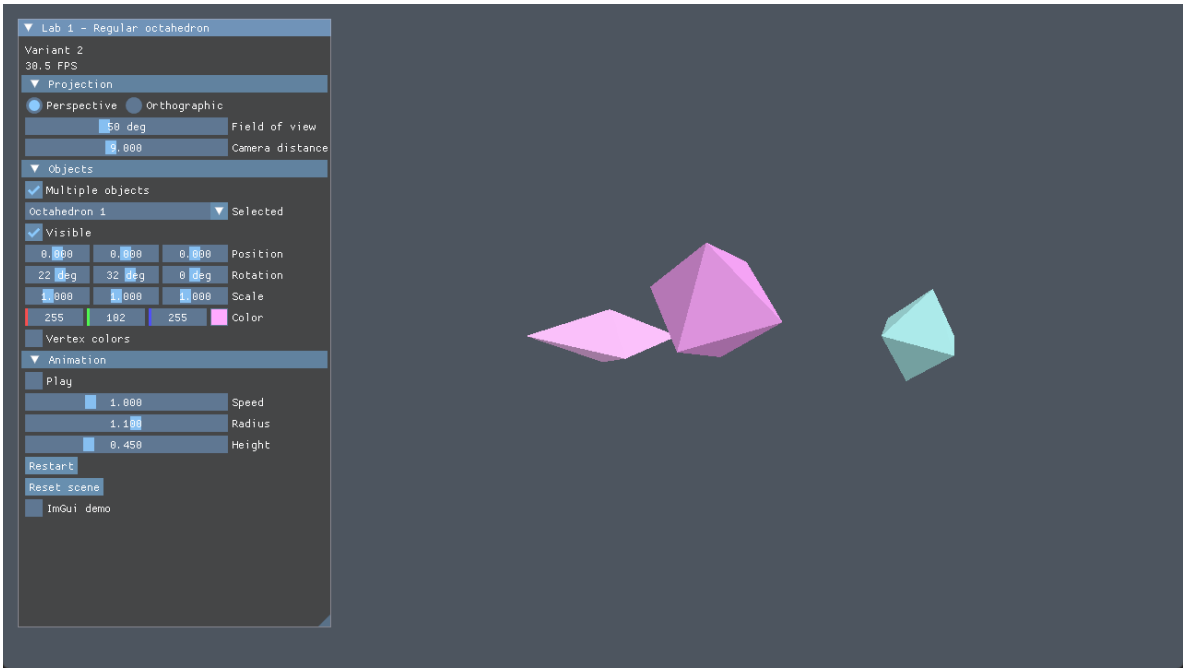


Рисунок 3 — Отключена процедурная окраска, изменены цвет первого объекта и масштаб второго. Анимация приостановлена, грани различимы благодаря освещению.

Получено приложение с интерактивной отрисовкой октаэдра, двумя видами проекции, управлением трансформациями, анимацией и окраской. Контрольные снимки демонстрируют работу нескольких объектов и независимое изменение их параметров.

Выводы

В ходе работы освоены представление трёхмерной модели треугольной сеткой, матрицы аффинных преобразований и проекции. Реализована передача данных в GPU через буферы и дескрипторы Vulkan, запись команд рисования и управление сценой с помощью ImGui. Проверены равенство рёбер и замкнутость октаэдра, диапазон глубины обеих проекций и порядок преобразований. Полученная реализация служит основой для дальнейшего развития камеры и освещения.